

1. What does REST mean in the context of Web APIs?

REST (Representational State Transfer) is an architectural style for building scalable web services. In Web APIs, it means treating data entities as resources identified by unique URLs (URIs) and manipulating them using standard HTTP methods in a stateless, client-server communication model.

Example:

In our Student API, `/api/students` identifies the collection of students, while `/api/students/5` identifies student with ID 5. Communication does not maintain session state on the server.

2. What is the difference between GET, POST, PUT, PATCH and DELETE?

Each HTTP verb defines a specific intent on a resource:

- **GET:** Retrieves a resource without modifying state (Safe and Idempotent).
- **POST:** Creates a new resource on the server (Neither safe nor idempotent).
- **PUT:** Replaces/updates an entire resource completely (Idempotent).
- **PATCH:** Applies partial modifications to specific fields of a resource.
- **DELETE:** Removes the resource from the system (Idempotent).

Example:

GET	<code>/api/students/1</code>	-> Returns Student #1
POST	<code>/api/students</code>	-> Creates a new Student
PUT	<code>/api/students/1</code>	-> Replaces all fields of Student #1
PATCH	<code>/api/students/1/status</code>	-> Updates only the IsActive status
DELETE	<code>/api/students/1</code>	-> Deletes Student #1

3. When should an API return 200, 201, 204, 400 and 404?

Standard HTTP status codes communicate execution outcomes predictably:

- **200 OK:** Standard success response returning data (e.g., GET or PUT).
 - **201 Created:** Resource successfully created, returning the new entity and a Location header (e.g., POST `/api/products`).
 - **204 No Content:** Action succeeded but returns no response body (e.g., DELETE).
 - **400 Bad Request:** Client error such as invalid payload, negative numbers, or validation failures.
 - **404 Not Found:** The requested resource ID does not exist in the system.
-

4. What is the difference between route parameters and query parameters?

Route parameters are part of the URL path used to identify a specific single resource. Query parameters appear after the ? character in key-value pairs and are used for filtering, searching, sorting, and pagination.

Example:

```
// Route Parameter (Identifies WHICH resource):  
GET /api/products/42  
  
// Query Parameters (HOW to filter or paginate the collection):  
GET /api/products?search=laptop&minPrice=500&page=1&pageSize=10
```

5. What is the role of a controller in ASP.NET Core Web API?

A controller acts as the entry point and coordinator for incoming HTTP requests. Its role is strictly to receive requests, validate ModelState, call the appropriate business service methods, and map the domain result into an HTTP response with appropriate status codes.

Example:

```
[HttpPost]  
public async Task<IActionResult> Create([FromBody] CreateProductRequest request)  
{  
    if (!ModelState.IsValid) return BadRequest(ModelState);  
    var result = await _productService.CreateProductAsync(request);  
    return CreatedAtAction(nameof(GetById), new { id = result.ProductId }, result);  
}
```

6. Why should we use DTOs instead of exposing models directly?

DTOs (Data Transfer Objects) decouple internal database entities from external client contracts. They prevent over-posting vulnerabilities (protecting sensitive internal properties), avoid circular reference serialization issues, and allow customized request and response payloads tailored to specific use cases.

Example:

In our Student API, CreateStudentRequest accepts only input fields needed for registration, while StudentResponse returns formatted data without exposing internal private keys or audit fields.

7. Why should business logic not stay inside the controller?

Putting business logic in controllers violates the Single Responsibility Principle (SRP) and Separation of Concerns (SoC). It creates tightly coupled code that cannot be unit-tested without mocking HTTP contexts and prevents reusing the same business rules across other interfaces (e.g., Background Jobs, CLI, or GraphQL).

Example:

Stock validation and inventory calculation logic lives in `ProductService.cs`, leaving `ProductsController.cs` lightweight and focused exclusively on HTTP routing.

8. What is Dependency Injection and why is it useful?

Dependency Injection (DI) is a design pattern where an object receives its dependencies from an external IoC container rather than creating them directly using `new`. It promotes loose coupling, enhances testability (enables mocking interfaces), and manages object lifecycles cleanly.

Example:

```
// In Program.cs: Register service with IoC container
builder.Services.AddSingleton<IProductService, ProductService>();

// In ProductsController.cs: Injected via constructor
public ProductsController(IProductService productService)
{
    _productService = productService;
}
```

9. Where can validation happen in a Web API?

Validation happens across multiple defensive layers:

1. **Model Validation (DTO Attributes):** *Uses Data Annotations (e.g., `[Required]`, `[Range]`, `[EmailAddress]`) to validate input shape automatically before action execution.*
 2. **Business Validation (Service Layer):** *Checks domain business rules against existing state (e.g., verifying if a product SKU already exists or if category ID is active).*
-

10. What is Swagger / OpenAPI used for?

Swagger (OpenAPI) is an interactive API specification tool that generates automated documentation for endpoints, request parameter types, DTO schemas, and response codes. It provides a built-in browser UI to test endpoints interactively during development without external software.

11. Why do we need Postman if Swagger already exists?

While Swagger is great for interactive documentation on a single running service, Postman provides advanced tooling:

- *Organizing organized test suites across multiple APIs into shareable Collections.*
 - *Managing environment variables (switching seamlessly between Localhost, Staging, and Production).*
 - *Automated integration testing with assertions and CI/CD automation.*
 - *Testing complex multi-step request workflows and edge failure cases.*
-

12. What makes an API response professional and predictable?

A professional API response provides consistent JSON envelopes across all endpoints, uses standard HTTP status codes, formats dates in ISO-8601 UTC, and returns structured, human-readable error messages for client diagnostics.

Example:

```
// Consistent Error Response Shape:
{
  "message": "Product with ID 99 was not found.",
  "statusCode": 404,
  "timestamp": "2026-08-31T20:30:00Z"
}
```

13. How would you implement search and filtering in an API?

By binding query parameters to a dedicated filter DTO in a GET action, then applying conditional LINQ query expressions (Where) in the service layer before materializing the collection.

Example:

```
public async Task<IEnumerable<ProductResponse>> GetFilteredAsync(ProductFilterQuery q)
{
    var query = _products.AsQueryable();
    if (!string.IsNullOrEmpty(q.Search))
        query = query.Where(p => p.Name.Contains(q.Search, StringComparison.
OrdinalIgnoreCase));
    if (q.CategoryId.HasValue)
        query = query.Where(p => p.CategoryId == q.CategoryId.Value);
    return query.Select(p => p.ToResponse());
}
```

14. Why is pagination important in APIs?

Pagination prevents server memory exhaustion and network latency by avoiding loading millions of database records in a single payload. It protects API performance and improves client responsiveness by fetching manageable chunks (e.g., 10 or 20 records) using `Skip()` and `Take()`.

15. What is the difference between UI, controller, service and data storage?

They represent distinct architectural layers:

- **UI (Presentation):** Frontend web/mobile application that the end-user interacts with.
 - **Controller:** HTTP Gateway that routes requests, validates input, and returns status codes.
 - **Service:** Core business engine that executes calculations, rules, and workflows.
 - **Data Storage:** The persistence layer (RAM in Phase 02, SQL Server/EF Core in Phase 03) storing data.
-

16. What should reviewers see in your commit history?

Reviewers should see an incremental, logical narrative of the project's evolution. Commits should be atomic, well-scoped, and follow Conventional Commits (e.g., `feat: add ProductsController`, `fix: correct price validation in DTO`, `docs: add Postman evidence`).

17. How do you prove that your API works without running it on the reviewer device?

*By delivering a comprehensive ****Evidence Pack**** containing:*

- Exported, ready-to-import Postman Collections and Environment JSON files.
 - High-resolution screenshots of Swagger UI schemas and Postman success/error responses.
 - Automated unit/integration test results and a clear, reproducible `README.md`.
-

18. How do you investigate an endpoint that returns 500?

1. Check server console logs and structured logging for stack traces and exception types.
 2. Identify the exact line of code causing the failure (e.g., `NullReferenceException`).
 3. Reproduce the exact request payload and headers in Postman/Swagger under the debugger.
 4. Add proper defensive checks, null-guards, or try-catch error handling to return clean 400/404 responses instead of unhandled 500 crashes.
-

19. Why should we not trust request data from users?

User input can be malformed, corrupted, or maliciously crafted (e.g., SQL Injection, Cross-Site Scripting, over-posting, negative prices, or privilege escalation). Never trusting raw input and strictly enforcing Data Annotation validations on DTOs ensures application security and data integrity.

20. Why is in-memory storage not enough for real applications?

In-memory storage is volatile: all data is lost whenever the server restarts or crashes. Furthermore, it cannot scale across multiple load-balanced server instances (data would be out of sync), and lacks relational database guarantees like ACID transactions, indexing, and backup capabilities. Real-world systems require persistent databases like SQL Server with EF Core.